

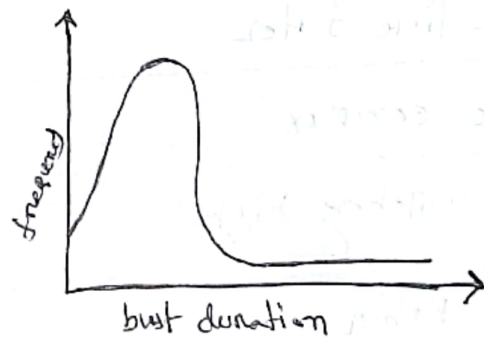
- CPU scheduling is technique in OS where the OS decides which process will use the CPU next.

When multiple processes are ready to run. the CPU can handle only one at a time. so the OS uses a scheduling algorithm to choose the best process.

- *** criteria
- maximize CPU utilization: How busy the CPU remains. Higher utilization better
 - minimize waiting time: Total time a process spend in the ready queue
 - minimize turnaround time: total time taken from process submission to completion
 - Improve ~~the~~ response time: Time from submission to first CPU ~~time~~ ^{response}
 - maximize throughput: No. of process complete per unit time.

- CPU burst time means the amount of time a process uses the CPU continuously for execution before it either

large number of short burst
small " " longer burst



The CPU scheduler select among the processes in ready queue and allocated a CPU core to one of them.

- decision:
1. switches from running to waiting state
 2. switches from running to ready state
 3. switches from waiting to ready
 4. Terminated.

~~xxx~~

- In Preemptive scheduling, the CPU can be taken away from running process before it finishes. interrupt allowed.

algorithm - Round Robin (RR)
shortest remaining time first (SRTF)
Priority (Preemptive)

- In non-preemptive scheduling, once a process gets the CPU, it keeps it until it finishes. no interrupt allowed.

First come first serve (FCFS)

Shortest job first (SJF)

Priority (non-preemptive)

Preemptive	non-preemptive
cpu control can be taken away	cpu control can't be taken
Response time faster	slower
more complex	simple
context switching high	context switching less
Round Robin	FCFS

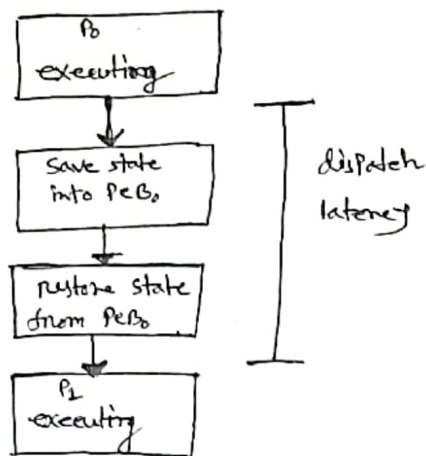
Dispatcher module gives control of the CPU to the process...
Selected by the CPU scheduler. execute m...

Fun:

context switching

Switch to user mode

Jump to correct location



$$\text{Turn around time} = \text{completion time} - \text{Arrival time}$$

$$\text{Waiting time} = \text{Turn around time} - \text{Burst time}$$

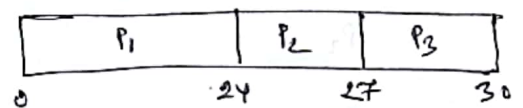
$$\text{Response time} = \text{First start time} - \text{Arrival time}$$

• Describe various algorithm :

• First come First serve : Process are executed in the order they arrive.
Queue system. FIFO

Process	Burst time
P ₁	24
P ₂	3
P ₃	3

arrive - P₁, P₂, P₃



waiting time, P₁=0, P₂=24, P₃=27

average time $0+24+27/3 = 17$

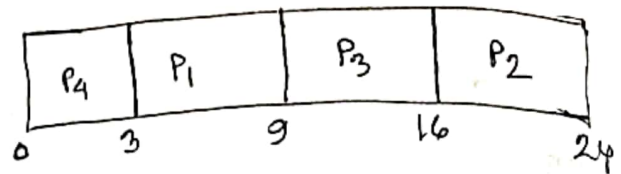
convoy effect = Short process behind long process.

Shortest job first : Process with smallest CPU burst time run first. ~~24/3 burst time run first~~

~~Preemptive~~ Preemptive version $\Rightarrow$ Shortest Remaining time first (SRT)

Process	Burst time
P ₁	6
P ₂	8
P ₃	7
P ₄	3

SJF scheduling chart:



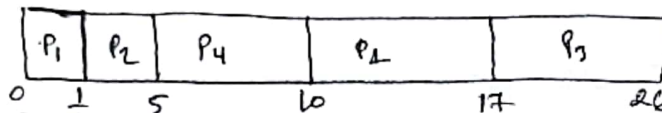
$$\text{Avg. time} = (0+3+9+16)/4$$

$$= 7$$

• Shortest remaining time first:

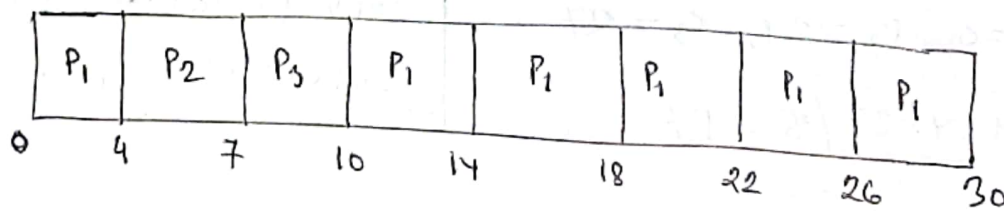
Process	Arrival time	Burst time	Wt
P1	0	8	
P2	1	4	
P3	2	9	
P4	3	5	

$$\text{Wt} = \frac{(10-1) + (1-1) + (17-2) + (5-3)}{4} = 6.5$$



Round Robin: Each Process gets a fixed time, after time ends, ^{quantum} Process get CPU.

Process P	Burst time	time quantum = 4
P1	24	
P2	3	
P3	3	



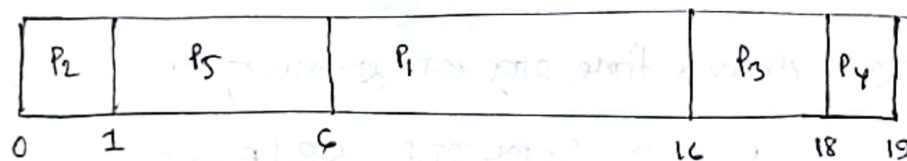
Priority scheduling: Each process has a priority, Higher priority

non dist. Problem - Starvation $\rightarrow$ Low priority processes may never execute

Solution - Aging $\rightarrow$ as time progresses increase the priority of the process.

Process	Burst time	Priority
P ₁	10	3
P ₂	1	1
P ₃	2	4
P ₄	1	5
P ₅	5	2

Priority



Average waiting time = 8.2

multi queue: Processes are divided into different queues. each

queue has its own scheduling algorithm. Prioritization based

highest

$\rightarrow$ Real time process $\rightarrow$

Priority = 0

T ₀	T ₁	T ₂	T ₃	T ₄
----------------	----------------	----------------	----------------	----------------

$\rightarrow$ System process $\rightarrow$

= 1

T ₅	T ₆	T ₇
----------------	----------------	----------------

$\rightarrow$ Interactive process $\rightarrow$

= 2

T ₈	T ₉	T ₁₀	T ₁₁
----------------	----------------	-----------------	-----------------

$\vdots$

lowest

$\rightarrow$ batch process $\rightarrow$

= n

T _x	T _y	T _z
----------------	----------------	----------------

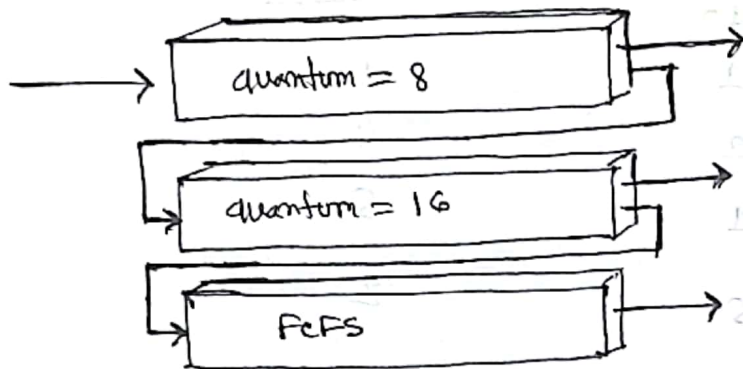
multilevel Feedback Queue: Processes can move between Queue.
Priority changes dynamically.

Priority changes dynamically.

Three queue: $Q_0 \rightarrow$ RR with time quantum : 8 millisecond
 $Q_1 \rightarrow$ RR time quantum 16 millisecond
 $Q_2 \rightarrow$ FCFS

$d_1 \rightarrow$ RR time quantum 16 milliseconds

Q2 $\rightarrow$ FDFS



Priority Same then Arrival time $\rightarrow$ RR or FIFO, ना $\rightarrow$ FIFO
" " " " Same then RR/FIFO

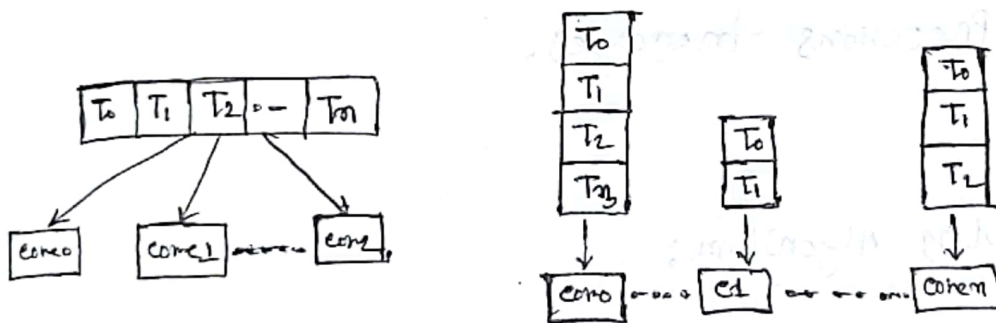
u 2 u Same 27 RR/FIFO

Thread Scheduling - Distinction between user level and kernel level thread.

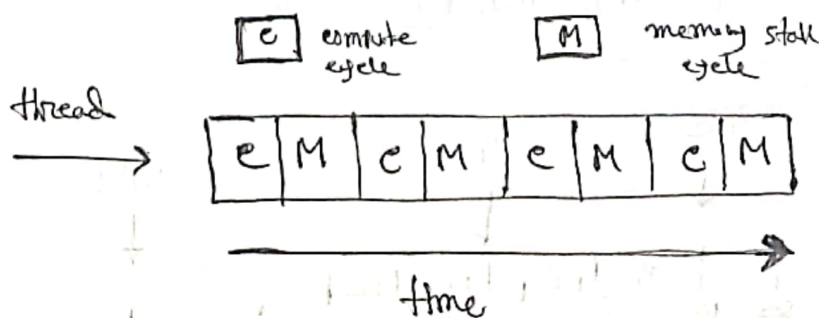
many to one, many to many, ~~the~~ model, thread. Library schedule user level threads to run on LWP. known as Process contention scope (PCS) since scheduling competition is within the process.

kernel thread scheduled onto available CPU is system contention scope (SCS) competition among all thread in system.

~~def~~ multiprocessor scheduling is the process of managing and assigning process across multiple CPUs. Asymmetric, symmetric



multicore scheduling - refers to scheduling processes on a single CPU that has multiple cores.

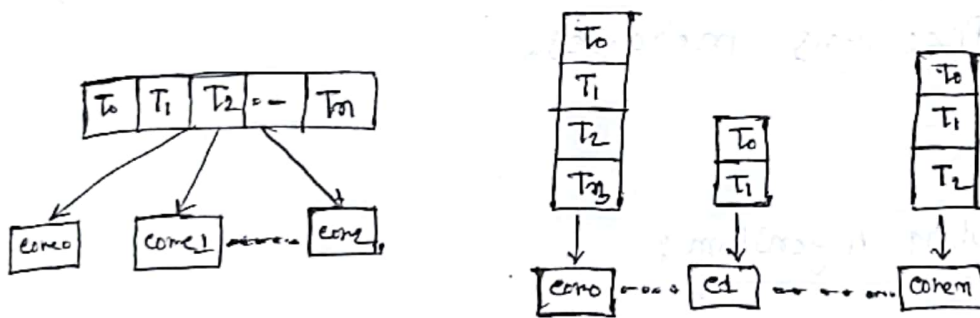


Thread Scheduling - Distinction between user level and kernel level thread.

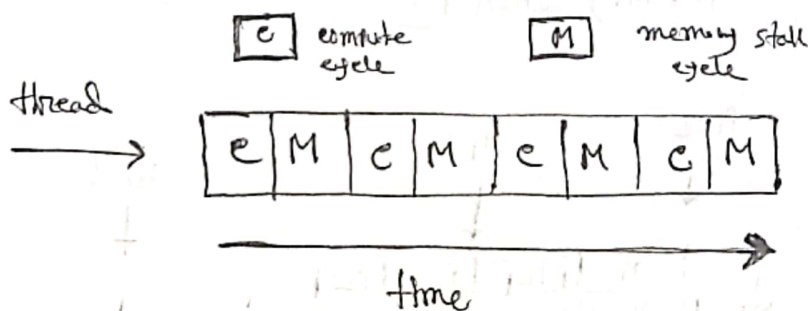
many to one, many to many, ~~the~~ model, thread. library schedule user level threads to run on LWP. known as Process contention scope (PCS) since scheduling competition is within the process.

kernel thread scheduled onto available CPU is system contention scope (SCS) competition among all thread in system.

~~✱~~ multiprocessor scheduling is the process of managing and assigning process across multiple CPUs. Asymmetric, symmetric



multicore scheduling - refers to scheduling processes on a single CPU that has multiple cores.



Issue

Load Balancing: work must be evenly distributed among all procs to avoid some being idle while others are overloaded.

Processor Affinity: A processor prefer to run on the same CPU to improve ~~each~~ cache performance.

Soft affinity: OS tries to keep process on same CPU.

Hard affinity: Process is fixed to a specific CPU.

Synchronization: Shared data between processors may cause race conditions and require proper synchronization mechanisms.

Scalability: Scheduling algorithm must perform efficiently as the number of processors increases.

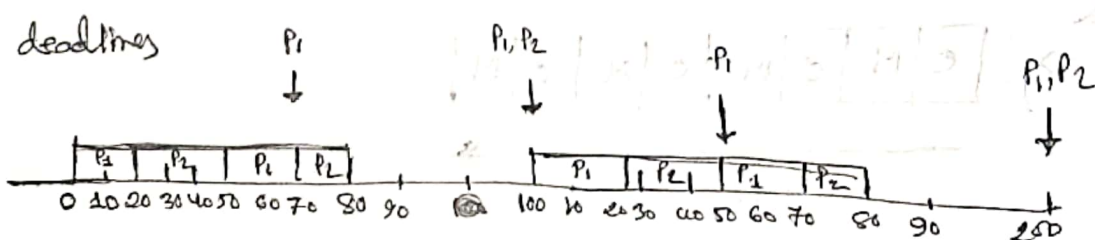
Real time scheduling Algorithm:

Real time system require task to be completed before deadline

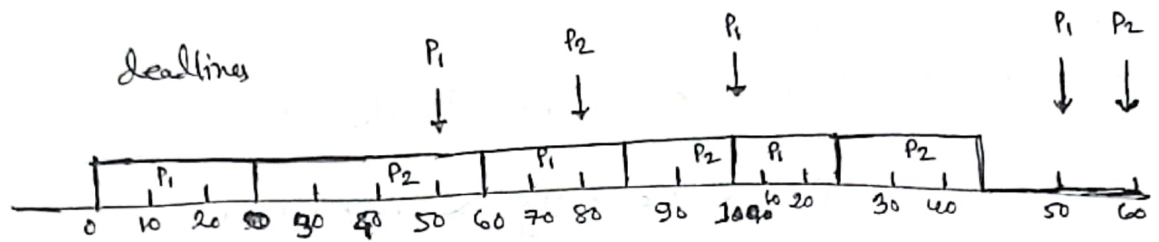
• Rate monotonic scheduling: Fixed priority scheduling. ~~Shorter~~

Shorter Period $\rightarrow$ higher Priority

P_1 assigned higher priority than P_2



Earliest Deadline first: Dynamic Priority scheduling, Task with the earliest deadline execute first.

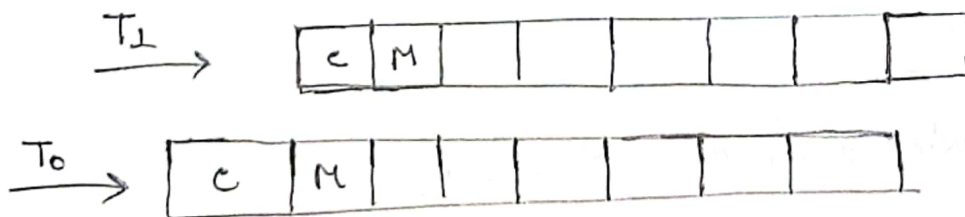


Least Laxity first (LLF): Based on laxity (~~deadline~~ deadline - remaining execution time)

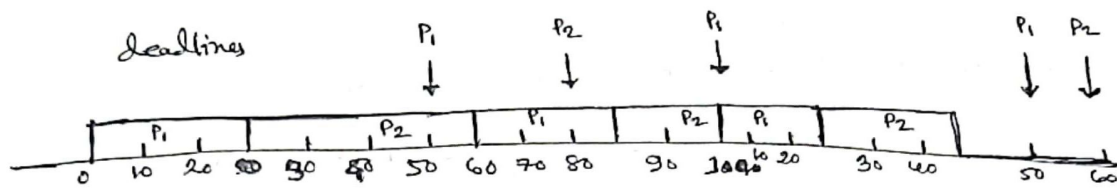
Task with least slack time gets highest priority.

multithreaded multicore system:

if one thread has memory stall, switch to another thread,



Earliest deadline first: Dynamic Priority scheduling, Task with the earliest deadline execute first.

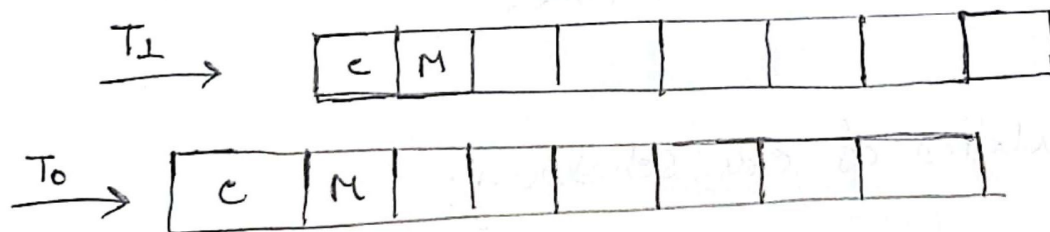


Least Laxity first (LLF): Based on laxity (~~slack~~ deadline - remaining execution time)

task with least slack time gets highest priority.

multithreaded multicore system:

if one thread has memory stall; switch to another thread.



Scheduling in OS:

Windows uses: Preemptive Priority based scheduling
Thread are assigned priorities
Higher priority thread except first

~~Solaris~~ Solaris: multilevel feedback queue scheduling
Different scheduling classes for different types of
Process.
Dynamic priority adjust

~~Solaris~~ Linux: completely fair scheduler

Aim for fair CPU time distribution among process
uses virtual run time.

• modeling and simulation of CPU scheduling:

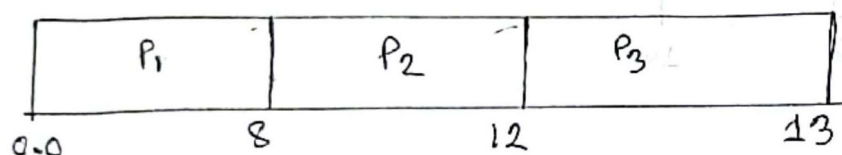
Deterministic: use a fixed set of input

Queueing model;

Simulation;

S.3 non-preemptive Scheduling: FCFS

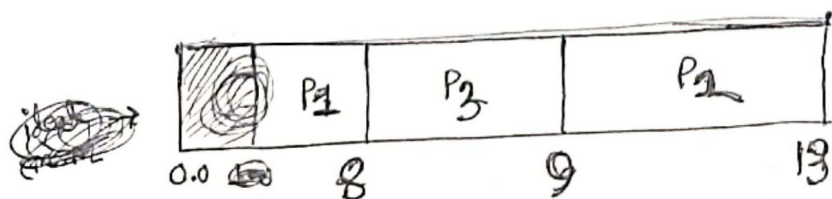
Process	Arrival time	Burst time	Completion time	Turnaround time	Waiting time	Response time
P ₁	0.0	8	8	8	0	0
P ₂	0.4	4	12	11.6	7.6	7.6
P ₃	1.0	1	13	12	11	11



(a) average turnaround time = $\frac{8 + 11.6 + 12}{3} = 10.53$

(b) SJF

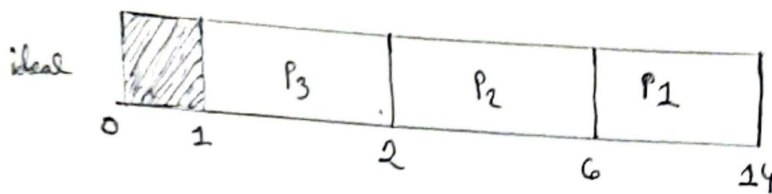
P	A	Bur	es	T	w	R
P ₁	0.0	8	8	8	0	0
P ₂	0.4	4	13	12.6	8.6	8.6
P ₃	1.0	1	9	8	7	7



avg. TAN = $\frac{8 + 12.6 + 8}{3}$
 $= 9.53$

(c)

P	A	B	C	T	W	R
P ₁	0.0	8	14	14	6	6
P ₂	0.4	4	6	5.6	1.6	1.6
P ₃	1.0	1	2	1	0	0



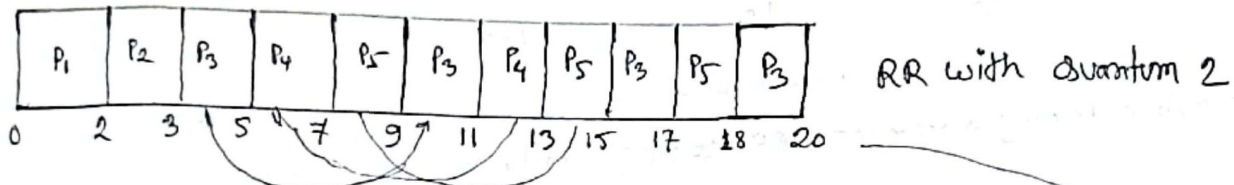
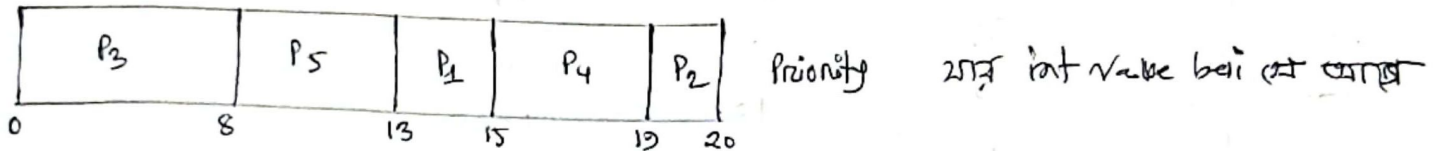
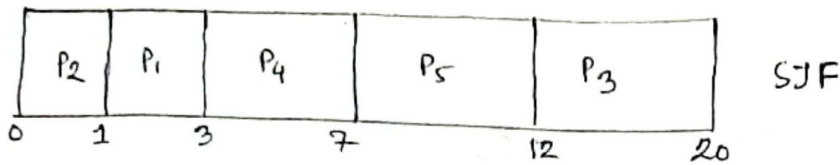
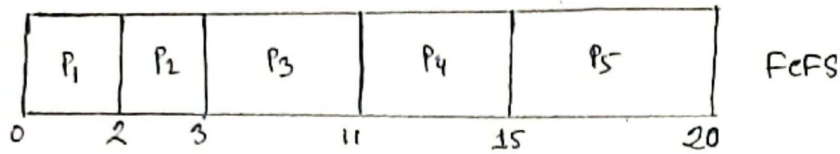
$$\text{avg TN.} = \frac{14 + 5.6 + 1}{3} = 6.87$$

- at time 0 only process P₁ is available, so it is selected. However if the CPU is kept idle from time 0-1 (assuming future knowledge), all process becomes available and shortest job can be selected first.

5.4

Process	Arrival time	Priority				
P ₁	2	2				
P ₂	1	1				
P ₃	8	4				
P ₄	4	2				
P ₅	5	3				

(a) All arrive at 0 time, P_1, P_2, P_3, P_4, P_5 so all 0 arrive time



(b) $TAN = (\text{completion time} - \text{Arrival time})$

Process	FCFS	SJF	Priority	RR
P_1	2	3	15	2
P_2	3	1	20	3
P_3	11	20	8	20
P_4	15	7	19	13
P_5	20	12	13	18

Last P_3 20th comp time

(c) $\text{Waiting time} = (\text{Turnaround time} - \text{Burst time})$

Process	FCFS	SJF	Priority	RR
P_1	0	1	13	0
P_2	2	0	19	2
P_3	3	12	0	12
P_4	11	3	15	9
P_5	15	7	8	13

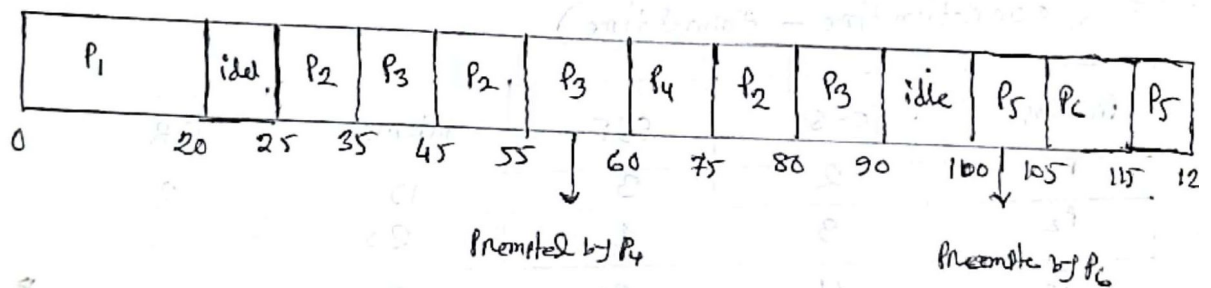
d. SJF has minimum wait time

5.5 ~~RR~~

Process	Priority	Burst	Arrival	
P ₁	40	20	0	
P ₂	30	25	25	
P ₃	30	25	30	
P ₄	35	15	60	
P ₅	5	10	100	
P ₆	6	10	105	

Preemptive Priority $\rightarrow$ same $\rightarrow$ RR (10 ms) higher priority preempt

(a)



(b) turn around time:

$$P_1 = (20 - 0) = 20$$

$$P_4 = (75 - 60) = 15$$

$$P_2 = (80 - 25) = 55$$

$$P_5 = (120 - 100) = 20$$

$$P_3 = (90 - 30) = 60$$

$$P_6 = (115 - 105) = 10$$

(c) waiting time:

$$P_1 = 0$$

$$P_4 = 0$$

$$P_2 = 30$$

$$P_5 = 10$$

$$P_3 = 35$$

$$P_6 = 0$$

Q) CPU utilization, total time = 120

$$\text{Idle} = (20 - 25) + (90 - 100)$$

$$= |-15| = 15$$

$$\text{Busy} = 120 - 15 = 105$$

$$\text{utilization} = (105/120) * 100 = \underline{87.5} \quad \underline{\text{Ans}}$$